

ORB project, summary of findings

Saverio Fontana

May 11, 2026

In this pdf I summarize all the findings on the ORB project. First of all clone the repo: [REPO].

Then here are what you will find:

1 final.Rnw

This is the file that allows to generate the report you are reading right now. Produce it by typing `make` on the console. I also added on Github the `data` folder. Inside it you will find two `.rds` files. Those will allow you to compile this script without re-running the data generation scripts.

You can remove those `.rds` files from Git once downloaded to keep the repo clean.

2 00.functions.R

In this script there are all the functions necessary to run the methods. Here is a quick overview.

2.0.1 `impute_missing_se(data, target_theta_col, target_se_col, n_col)`

Impute missing standard errors for one outcome using the reported studies following the formula from Copas.

2.0.2 `run_univariate_imputation(data, theta_col, se_col, m = 1000, new_version = FALSE, method_re = "REML")`

This generates multiple imputations for ONE partially missing outcome under MAR assumption.

It follows a few steps:

- Reorders data as reported first, unreported second.
- Fits a naive random-effects meta-analysis on only the reported studies with `metafor::rma`.

- Builds a total covariance matrix Σ .
- Partitions Σ into reported and unreported.
- Computes conditional means and covariance for missing effects and draws m imputations with `MASS::mvrnorm`.

Note that selecting `new_version = TRUE` uses the formula:

$$\Sigma_j = \text{diag}(\sigma_{1j}^2 + \tau_j^2, \dots, \sigma_{Kj}^2 + \tau_j^2) + \text{SE}(\hat{\theta}_{j,\text{MA}})^2 J_K \quad (1)$$

while if `new_version = FALSE`, then τ^2 on the diagonal is omitted.

2.0.3 `adj.univariate(mi.results, delta = 0.5, select.type = "zscore", method.re = "REML", track.ess = TRUE)`

Applies ORB importance sampling on the imputed datasets outputted by the function `run.univariate.imputation`.

What it does is:

- For each imputed dataset, computes log-weights from either:
 - z-scores (θ/SE) , or
 - raw effects (θ) .
- Refits `rma` for each completed dataset.
- Normalizes weights using log-sum-exp (result is the same as using classic normalization, but more stable).
- Computes weighted estimate and variance decomposition using paper formulas .

If `track.ess` is `TRUE`, then an additional column is added to the final df containing the Effective Sample size to evaluate the IS weights, computed as $\frac{1}{\sum w_{norm}^2}$. The distribution of ESS could be added (or not) to the final tables in the paper as a diagnostic for IS.

2.0.4 `run.bivariate.imputation(data, theta.cols, se.cols, rho.w = 0.4, m = 1000, new_version = FALSE, method.re = "REML")`

This Generates imputations under MAR with two outcomes modeled jointly, the process is similar to the univariate.

```
2.0.5 adj_bivariate(mi_results, delta = 0.5, select_type = "zscore",
  method.re = "REML", track.failed.proportion = TRUE, track.ess
  = TRUE)
```

Applies ORB importance sampling on the imputed datasets outputted by the function `run_bivariate_imputation`.

Note that I compute only one weight per imputation, and it is used for both outcomes! (The simulation study is not affected by this choice, but the topiramate example yess)

Using `track.failed.proportion = TRUE`, we can also track the proportion of failed calls to `rma.mv`.

```
2.0.6 generate_bivariate_ma(K = 12, theta = c(0.4, 0.4), tau2 = c(0.06,
  0.06), rho_b = 0.4, rho_w = 0.4, n_arm = 50)
```

Simulate complete bivariate meta-analysis data without any missingness.

```
2.0.7 impose_orb(data, p1 = 0.4, delta_sim = 0.5, select_type = "zscore",
  orb.se = TRUE)
```

Impose ORB on Outcome 1 in the simulated data obtained from `generate_bivariate_ma`.

It finally returns a data frame with ORB-induced missingness for Outcome 1.

Note: at the beginning I wasn't sure wheter also the SEs were omitted, so I added the parameter `orb.se`, if `TRUE`, then also SEs are omitted.

01.topiramate_data.prep.R

To create the topiramate df.

02.topiramate_fit.R

To fit what we obtained from `01.topiramate_data.prep.R`, it allows to compute Table 2 of the paper.

Note: I fit it using the old method `new_version = FALSE` for the covariance matrix, but it can be easily changed for the new version.

03.exploratory.check.R

In this script I ran a small simulation study: I generated data with K studies, true effects $\theta = (0.4, 0.4)$, between-study heterogeneity τ^2 , ρ_B and ρ_W fixed at 0.4. Then Outcome 1 is made missing with strength δ and $p_1 = 0.4$ expected missing rate, using z-score selection. Missing SEs are imputed using Copas formula.

Scenarios evaluated: 12 combinations of $K \in \{6, 25\} \times \tau^2 \in \{0.06, 0.36\} \times \delta \in \{0, 0.5, 1\}$, each with 1000 Monte Carlo replicates. NOTE that the model is always correctly specified (e.g. if I generate data using $\delta = 0.5$, then also the fit is made with $\delta = 0.5$)

Methods compared per each replicate:

- Baseline: full data REML and PM (like an oracle, before imposing ORB)
- Naive: unadjusted estimates, obtained from reported studies only (old and new Sigma versions, uni and biv)
- Adjusted univariate: old and new Sigma, REML and PM, at $M \in \{50, 200, 1000\}$
- Adjusted bivariate: old and new Sigma, at $M \in \{50, 200, 1000\}$

I also saved the `ess` for each method that involve Importance Sampling. I wanted to use it as diagnostic but now I am not sure anymore, can be useful in some discussion part for the paper, but can also be ignored.

Figure 1 shows the Bias across different scenarios (columns) and different methods (rows) when $M \in \{50, 200, 1000\}$. As we could expect, as delta increases, the bias also increases. The difference in bias between 50 and 200 imputations is notable, but the difference in bias between 200 and 1000 seems negligible. Interestingly, in the case of $K = 6$ for univariate (new), $M = 1000$ has an higher bias (in absolute value) than $M = 200$ and 50.

Note that if you wanna see the results with PM instead of REML, you just need to comment out a line of code in the `.Rnw` file, but the results are quite similar so I decided to not report them here.

Maybe if the code takes too long to run all the 30000 (and more) simulations, also using $M = 200$ can work?

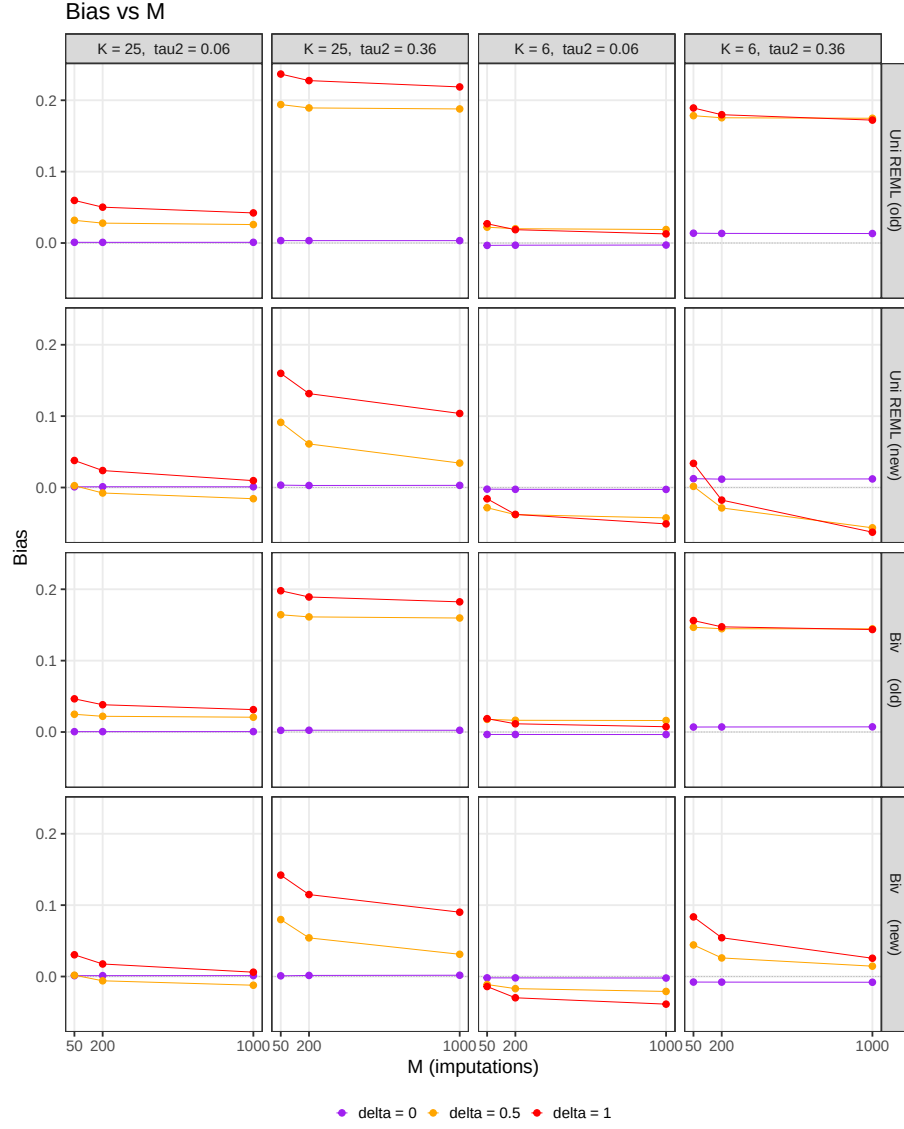


Figure 1: Bias by varying M across scenarios

Figure 2 reports the bias across methods. Each column represents a δ value, the first two rows have $K = 25$, while the 3rd and 4th have $K = 6$. Importantly, when $\delta = 0$, all methods seems to provide unbiased results. The naive methods provides the worst performances consistently across all scenarios. The behavior of the other methods change across scenarios but the new one seems to provide results close to zero bias more frequently.

Results obtained by using also PM for τ are in 3.1.

NOTE: if you wanna see the results with $M = 1000$, you just have to change one line of code in the .Rnw.

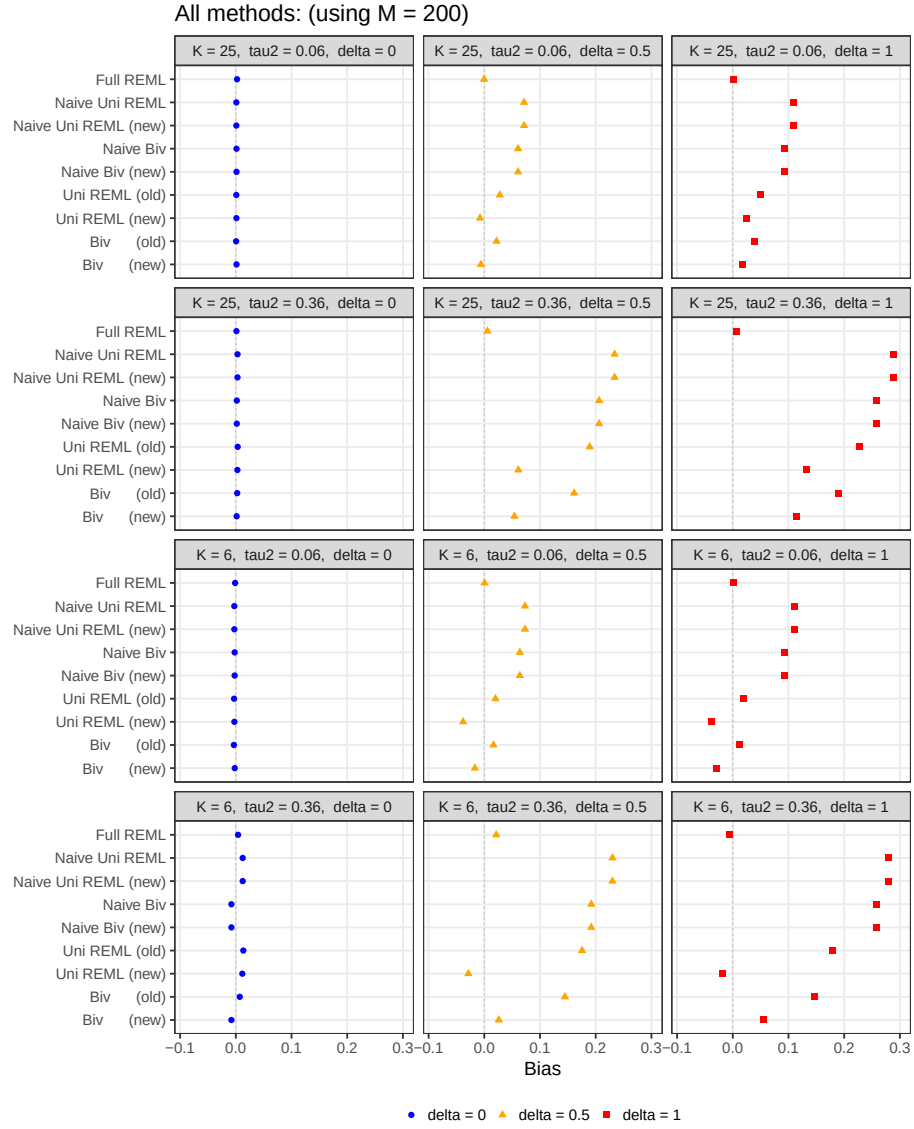


Figure 2: Bias comparing different methods

2.1 Inspection of the possible failures

Here is all the type of failures that can happen in the simulations:

- ERROR 1. `impose_orb()` produces no missingness (since it is based on `rbinom()`), when by chance no studies are set to NA. Intuitively more common when K small.
- ERROR 2. `impose_orb()` produces too much missingness. Happens when fewer than 2 studies are left and the other are all set to NA.
- ERROR 3. `rma/rma.mv` fails to converge inside `adj_univariate/adj_bivariate`. I am only tracking it in the biv case in the column `'failed.proportion'`

Results: We have a total of 12 scenarios, each run 1000 times. This mean that in total we have 12000 simulations.

We observe a total of 476 (3.97% out of the total simulations) ERROR 1

And a total of 137 (1.14% out of the total simulations) ERROR 2

Table 1: Proportions of Error Types 1 and 2 by scenario

K	tau2	delta	n_total	n_err_type1	pct_type1	n_err_type2	pct_type2
6	0.06	0	1000	40	4.00%	38	3.80%
6	0.06	0.5	1000	85	8.50%	24	2.40%
6	0.06	1	1000	114	11.40%	12	1.20%
6	0.36	0	1000	41	4.10%	37	3.70%
6	0.36	0.5	1000	97	9.70%	14	1.40%
6	0.36	1	1000	98	9.80%	12	1.20%
25	0.06	0	1000	0	0.00%	0	0.00%
25	0.06	0.5	1000	0	0.00%	0	0.00%
25	0.06	1	1000	0	0.00%	0	0.00%
25	0.36	0	1000	0	0.00%	0	0.00%
25	0.36	0.5	1000	0	0.00%	0	0.00%
25	0.36	1	1000	1	0.10%	0	0.00%

Looking more in depth, we notice that out of the total, almost every ERROR 1 and ERROR 2 appear when $K = 6$. This mean that Bias estimates for $K = 6$ are based on approximately 950 replicates rather than 1000, which is still acceptable I'd say.

Here we instead report the proportion of failed `rma` calls (ERROR 3)

Table 2: Proportion of Error Number 3

K	tau2	delta	Biv old (%)	Biv new (%)
25	0.06	0	0.01%	0.01%
6	0.06	0	0.01%	0.01%
25	0.36	0	0.00%	0.00%
6	0.36	0	0.00%	0.00%
25	0.06	0.5	0.01%	0.02%
6	0.06	0.5	0.02%	0.02%
25	0.36	0.5	0.00%	0.00%
6	0.36	0.5	0.00%	0.00%
25	0.06	1	0.01%	0.02%
6	0.06	1	0.02%	0.01%
25	0.36	1	0.00%	0.00%
6	0.36	1	0.00%	0.00%

3 04.simulation_study.R

In this script I tried to give a blueprint for how to use these functions for the simulation study. There is an important note, since we saw that sometimes the `impose_orb` function can give a full dataset with no ORB. I think there are 3 ways to solve this:

- Leave it like I did. My reason is that, if for example $p_1 = 0.2$, then maybe sometimes the fraction of unreported for that specific draw is 40%, but maybe some other time is zero. If we truncate the distribution by discarding the draws with $p_1 = 0$, I am not actually sure it is mathematically sound
- use a repeat loop inside the lapply, to make it sample another df
- Change the `impose_orb` function to redo the ORB in the case that it creates no missings

NOTE: I never ran the script fully, so I am not sure it wil work without any problems

3.1 Some Final comments

- I left a note inside `run_bivariate_imputation()` about this, but I rewrite it: When we only have a few studies, the estimate of Ψ is probably super innacurate. I see two ways to solve this: either we fall back to `struct = "VC"` to make it easier for REML to estimate the matrix, since it doesnt have to estimate the off diagonals (p_B), or we fall back to the original formula without $\hat{\Psi}^2$ ¹. My only point is that from the mathematical

¹Note that this problem of few studies can come from also in the univariate case for $\hat{\tau}^2$

standpoint, including $\hat{\Psi}$ (or $\hat{\tau}^2$) is the correct version. Here is a small conceptual proof.

Proof. The old formula for the covariance matrix comes from the Fixed effect decomposition, where we assume tha for study i , $\hat{\theta}_i = \hat{\mu} + \epsilon_i$. Here, $Var(\hat{\mu}) = SE(\hat{\theta}_{MA})^2$ and $Var(\epsilon_i) = \sigma_i^2$.

Since the overall mean uncertainty and the sampling errors are independent, then:

$$Var(\hat{\theta}_i) = Var(\hat{\mu} + \epsilon_i) = Var(\hat{\mu}) + Var(\epsilon_i)$$

$$Var(\hat{\theta}_i) = SE(\hat{\theta}_{MA})^2 + \sigma_i^2$$

. These are the diagonal elements of the covariance matrix.

For the off diagonal:

$$Cov(\hat{\theta}_i, \hat{\theta}_k) = Cov(\hat{\mu} + \epsilon_i, \hat{\mu} + \epsilon_k) = Var(\hat{\mu}) = SE(\hat{\theta}_{MA})^2$$

But we are only fitting Random Effects, so:

The estimated value $\hat{\theta}_i$ for study i is decomposed as: $\hat{\theta}_i = \hat{\mu} + \delta_i + \epsilon_i$

Where we have a similar result as before, but we also have the random effect $\delta_i \sim N(0, \hat{\tau}^2)$.

Since the overall mean uncertainty, the random effects, and the sampling errors are independent, then:

– **Diagonal elements:**

$$Var(\hat{\theta}_i) = Var(\hat{\mu} + \delta_i + \epsilon_i) = Var(\hat{\mu}) + Var(\delta_i) + Var(\epsilon_i)$$

$$Var(\hat{\theta}_i) = SE(\hat{\theta}_{MA})^2 + \tau^2 + \sigma_i^2$$

– **Off-Diagonal elements:**

$$Cov(\hat{\theta}_i, \hat{\theta}_k) = Cov(\hat{\mu} + \delta_i + \epsilon_i, \hat{\mu} + \delta_k + \epsilon_k) = Var(\hat{\mu}) = SE(\hat{\theta}_{MA})^2$$

Therefore the final formula would be:

$$\Sigma_{RE} = \text{diag}(\sigma_1^2 + \hat{\tau}^2, \dots, \sigma_K^2 + \hat{\tau}^2) + SE(\hat{\theta}_{MA})^2 \mathbf{J}_K$$

And we can easily generalize to the bivariate case

□

Additionally I think using the hat conveys more clearly the idea that those are estimated parameters, but this is just a notation decision.

- in each script I used `RNGkind("L'Ecuyer-CMRG"); set.seed(1)`, I don't know if it should be mentioned in the paper.

Appendix

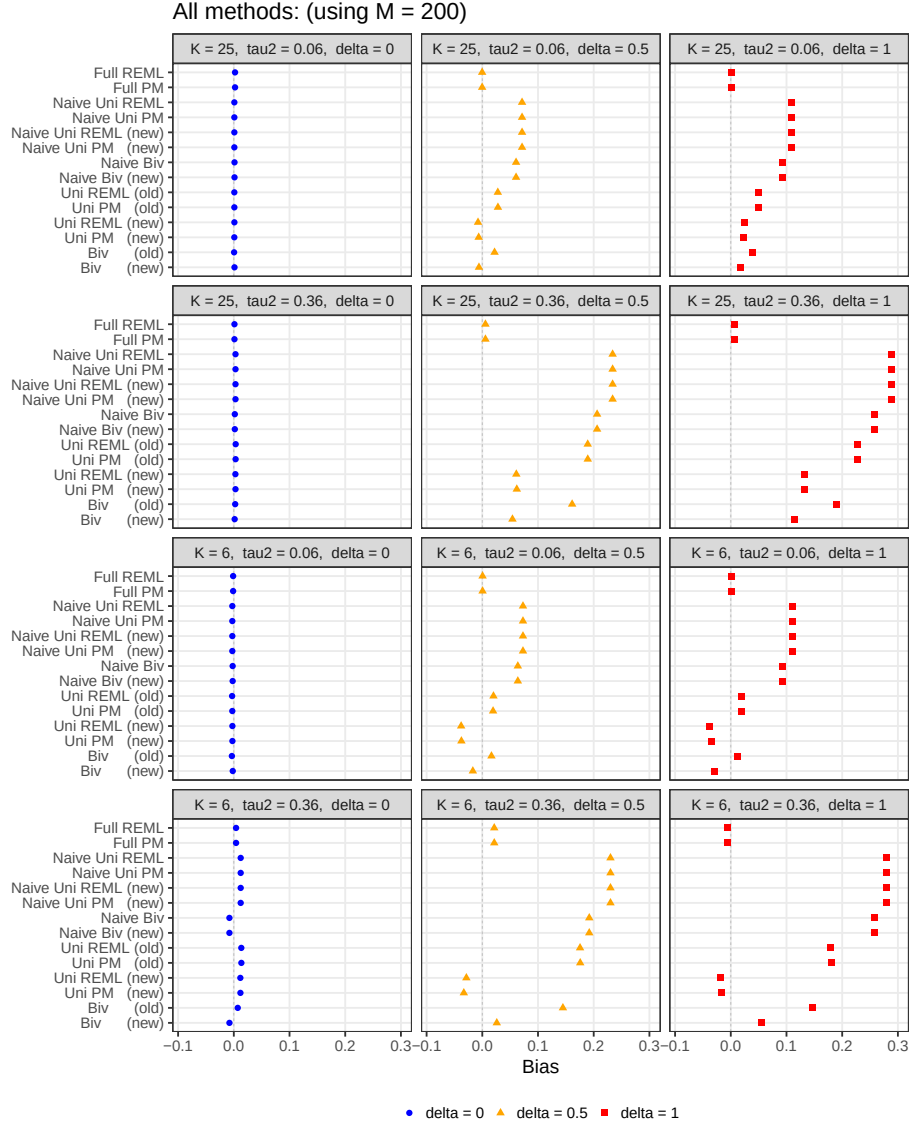


Figure 3: Bias by varying M across scenarios, COMPLETE VERSION